State-Gated Promotion: Auditable Repair of Multilingual LoRA Adapters via Frozen Deployment Gates

Byoungsang Lee[1,2] **Yunchul Kim**[1] **Youmin Shim**[1] **Chaewon Kwak**[1] **Jung Heon Lee**[1,3]
[1]School of Advanced Materials Science and Engineering, Sungkyunkwan University (SKKU), Suwon 16419, Korea
[2]MoonTechnology, 3F, 29-5, World Cup Buk-ro 48-gil, Mapo-gu, Seoul 03927, Korea
[3]Department of MetaBioHealth, Sungkyunkwan University (SKKU), Suwon 16419, Korea
jhlee7@skku.edu

Abstract

A LoRA adapter can win on validation loss yet emit unparseable JSON, drift into the wrong script, or answer in a language the session never activated — breaking the deployed interface even though scalar metrics look fine. We introduce *state-gated promotion*: a small set of automatic gates that name the deployment states an interface must preserve, freeze them before model selection, and turn each gate failure into targeted repair data. On a four-language KO/RU/FR/EN deployment with Gemma 4 E2B, a gate-aware repair curriculum lifts JSON-schema pass rate from **0% to 95.8%** and session-language routing from **0% to 91.7%** against a no-policy ablation matched on base, hyper-parameters, audit, and translation corpus but not on data volume, with no held-out loss penalty. The same *stock* failure replicates on four other instruction-tuned bases (Gemma 4 E4B, Qwen 2.5 3B, Llama 3.2 3B, Phi-3.5 mini), and applying the repair recipe to the three non-Gemma bases recovers schema compliance to $\geq$91.7% in every case — evidence that the failure is a property of the deployment interface, not of any one base model. Code and audit capsule: https://github.com/carryer123/gemma4-trilingual-family.

1 Introduction

Consider a multilingual chat session that declares Korean, Russian, and English active. A LoRA-fine-tuned adapter answers in French, returns prose where the parser expects a JSON object, and silently drops the `age_band` field. Validation loss is fine. BLEU is fine. A downstream parser still crashes, and the interface fails. This pattern is what we call an *interface-state failure*: the model satisfies the metrics it was selected on, but fails the runtime contract the deployment actually requires — script, schema, active-language routing, age/register policy.

The standard remedy is to filter on more scalar metrics or to reject offending adapters post hoc. We argue both miss the point. A scalar score, however refined, summarises behaviour in aggregate; the deployment cares whether *specific named states* hold. And rejecting an adapter throws away the diagnostic signal — which state failed, on which probe — that could have driven a targeted repair.

We propose *state-gated promotion and repair* (Figure 1). A small set of automatic gates, each named after a deployment state, is frozen before model selection. Each gate emits a GREEN/ AMBER/ RED action: GREEN admits, AMBER triggers documented repair or scoped waiver, RED blocks promotion until retraining, rollback, or an explicit deployment-specification change is re-audited. Critically, gate failures are not just rejection signals — they are training data. The same probes that diagnose a missing JSON key define the slice of repair examples a downstream curriculum needs.

We instantiate this loop on a four-language KO/RU/FR/EN deployment with two sessions (KO/RU/EN and KO/FR/EN) and four gates: G1 (output-card structure and age policy), G2 (script-state discipline), G3 (JSON/schema validity), G4 (session-language routing). Against a no-policy ablation (matched on base, hyper-parameters, audit, and translation corpus but not on data volume) on Gemma 4 E2B, a gate-aware *policy+family* curriculum recovers G3 from 0% to 95.8% and G4 from 0% to 91.7% with no held-out loss penalty. Applying the same recipe to three additional instruction-tuned bases (Qwen 2.5 3B, Llama 3.2 3B, Phi-3.5 mini) recovers G3 to $\geq$91.7% on every base, and Gemma 4 E4B reproduces the stock failure. We call an adapter that looks acceptable on loss yet collapses on the gates a *false-green* adapter; the no-policy arm produces exactly that, and the repair loop is what catches and fixes it. The contribution is the loop itself — frozen gates that simultaneously

diagnose failure, define repair data, and re-evaluate the result — together with cross-base evidence that the failure pattern is a property of the deployment interface, not of any one base model.

2 Related Work

This work sits closest to three lines of prior art: adapter selection, behavioral evaluation, and structured-output reliability.

Adapter selection. LoRA and other PEFT methods (Hu et al., 2022; Houlsby et al., 2019; Liu et al., 2022; Dettmers et al., 2023; Pfeiffer et al., 2020a,b) make it cheap to train many candidate adapters, shifting the deployment problem from "can we fine-tune?" to "which fine-tuned artifact is eligible to promote?" Standard selection signals — held-out cross-entropy, BLEU (Papineni et al., 2002), chrF (Popović, 2015), sacreBLEU (Post, 2018), broader benchmark suites (Wang et al., 2018, 2019; Liang et al., 2023; Hendrycks et al., 2021; Srivastava et al., 2023), and multilingual MT tracks (Goyal et al., 2022; NLLB Team, 2022; Kocmi et al., 2024) — summarize task behavior in aggregate. They do not name *deployment states*. This paper therefore treats scalar metrics as necessary but insufficient filters, not as deployment certificates.

Behavioral evaluation. Aggregate scores have long been shown to hide subgroup and task-state failures (Ribeiro et al., 2020; Gebru et al., 2021; Bender and Friedman, 2018; Mitchell et al., 2019; Bommasani et al., 2021). Multilingual coverage shows the same pattern in a different form: encoders and generators (XLM-R, XGLM, Aya, Gemma) and audits of multilingual web corpora (Conneau et al., 2020; Lin et al., 2022; Üstün et al., 2024; Gemma Team, 2024; Khashabi et al., 2020; Kreutzer et al., 2022) all reveal that broad coverage scores can mask narrow interface failures in lower-resource directions. Our case is narrower. We do not build a general multilingual benchmark; we ask how a small team should document promotion decisions when a deployment has explicit non-negotiable states. The resulting audit is operational — it records raw generations, scorer versions, gate states, and the triggered action — and its primary output is not a rank but a pipeline decision: *eligible*, *inspect/repair*, or *block*.

Structured-output reliability. JSON validity, function calling, and constrained decoding (Bray, 2017; Scholak et al., 2021; Willard and Louf, 2023; Beurer-Kellner et al., 2023) all target the same surface our G3 gate inspects. G3 is not a semantic task score; it is an *interface-discipline* gate — the model must emit parseable JSON with required keys, correct types, allowed enum values, and no extra keys. A model can be semantically helpful yet fail G3 if the parser cannot consume the output, and conversely passing G3 does not certify semantic correctness. We keep these two claim sides separate throughout the paper.

3 State-Gated Promotion and Repair

The workflow has three moving parts: a versioned audit tuple (what is recorded for every candidate), four named gates (the deployment states the interface must preserve), and three pipeline actions (GREEN/ AMBER/ RED).

Audit tuple. For every candidate adapter a we store

$$\text{audit}(a) = \{L, Q, G_1, \ldots, G_4, R, A\}(a),$$

where L is held-out loss, Q an optional task metric, G_i the four deployment gates, R the raw generations plus scorer version, and A the resulting pipeline action. Two design choices matter. The gates are *not* collapsed into a single leaderboard number — each must pass on its own. And the raw generations R are retained, because they are exactly the data a downstream repair curriculum will need.

Gates. Each gate names a state the deployed interface must preserve: G1 — output-card structure and age policy; G2 — script-state discipline; G3 — JSON/schema validity; G4 — session-language routing.

G1 (output-card structure and age policy). In our setting, an *output card* is the structured family-facing response the model is asked to produce: a body of text with target-age and active-language metadata, plus optional next-action fields. G1 checks that this card is usable at the body level — the model produces a card the interface can render, and the body sentences obey an age-policy budget (sentence length and vocabulary depth scaled to the target age). G1 is intentionally looser than G3 on top-level schema validity; strict required-key and field-type checks belong to G3. The four-language audit uses 24 prompts, split into structural and age-policy subchecks.

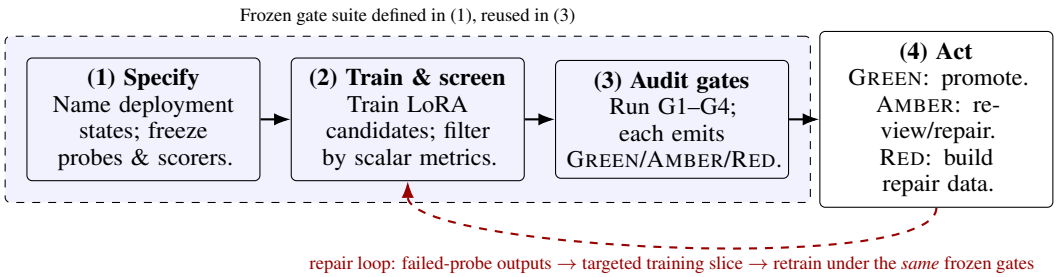

Figure 1: State-gated promotion and repair loop. The dashed-blue outline marks the frozen gate suite shared by step (1) and step (3). The red dashed arrow is the repair loop: a RED action in (4) converts the failed probes' raw outputs into a targeted repair slice that re-enters (2)'s training data; the resulting adapter is re-audited under the *same* frozen gates.

Gate	Deployment state	Evidence in this paper
G1	Output-card structure and age policy	24 four-language prompts; split into structural and age-policy subchecks.
G2	Cross-script state discipline	52-probe KO/RU rerun; 40 four-language script-state prompts.
G3	JSON/schema validity	80-probe KO/RU schema rerun; 24 four-language schema prompts.
G4	Session-language routing	24 four-language prompts checking inactive-language leakage.

Table 1: Operational gates used for promotion and repair. The four-language result is evaluated on G1–G4; any non-automated dimensions of deployment quality are outside the empirical claim of this paper.

G2 (script-state discipline). G2 is a Unicode-block script-state check, *not* a phonetic transliteration-quality metric. The expanded audit uses 52 prompts — four directions (KO→Cyrillic, RU→Hangul, KO→Latin, RU→Latin) with 13 lexical surfaces each. A response passes when the target script (Hangul, Cyrillic, or Latin) accounts for $\geq$85% of non-whitespace characters and no other tracked script exceeds 10%. The check is intentionally shallow: it does not ask whether the output is phonetically correct, only whether the adapter is in the requested script state at all.

G3 (JSON/schema validity). The expanded G3 audit uses 80 automatic schema prompts across four groups: object cards, intent routing, age/register rewrites, and tool-call JSON. The scorer implements the deployment parser contract — extract one JSON object, then check required keys, types, enum values, and no-extra-key constraints where declared. G3 is therefore an extractable-schema gate rather than a strict raw-JSON-only gate; a stricter pipeline could replace it with constrained decoding or a raw-only parser. *G3 checks schema discipline, not semantic correctness.*

G4 (session-language routing). G4 verifies that a session emits only the languages it has activated. In our setting, a KO/RU/EN session must not leak French and a KO/FR/EN session must not leak Russian. The four-language audit uses 24 prompts that pair an active-language declaration with a request an under-routed adapter would answer in the inactive language.

State	Pipeline action
GREEN	Eligible for deployment candidate status; log raw outputs, scorer version, gate version, and selected artifact hash.
AMBER	Automatic deployment is blocked; inspect raw outputs, label errors, add a minimal targeted repair slice or document a scoped waiver, then rerun failed gates.
RED	Promotion is blocked; rollback, retrain, or change deployment specification. A RED state cannot be silently waived.

Table 2: Audit states are actions, not just labels. AMBER and RED are not overrides: AMBER permits documented repair or scoped waiver plus failed-gate rerun; RED requires rollback or retraining unless the deployment specification itself changes and the full audit is rerun.

Example per-probe thresholds for the script and schema gates (G2-52, G3-80) are summarised in Table 3; the four-language audit uses the same band structure tuned to its smaller probe counts. They are *engineering triage rules*, not calibrated precision/recall cutoffs: we report sensitivity instead of

a single cutoff so the deployment owner can choose stricter or looser bands. The four-language audit (24 probes each on G1, G3, G4; 40 on G2) uses per-gate bands of the same form as Table 3 but tuned for the smaller probe counts; the realized thresholds and resulting Free/Deployed actions are logged in the released action JSON. "Free" vs "Deployed" in Table 7 distinguishes the raw adapter from one behind deterministic JSON/active-language enforcement, which can rescue G3/G4 but not G1. What the paper requires is reproducibility — the threshold version, raw outputs, and resulting action state must all be logged.

4 Experiment: Four-Language Curriculum Repair

We test whether the gates can both *diagnose* a failure and *drive* its repair when the curriculum is gate-aware from the start. The same recipe is then applied to three additional instruction-tuned base models (Qwen 2.5 3B, Llama 3.2 3B, Phi-3.5 mini) to check whether the failure pattern is base-specific; Gemma 4 E4B is included as a stock-only control.

4.1 Setup

We train Gemma 4 E2B LoRA adapters for a four-language KO/RU/FR/EN deployment with two sessions, KO/RU/EN and KO/FR/EN. The deployment supports four languages globally but activates at most three per session, so G4 session routing is deployment-critical: a KO/RU/EN adapter must not leak French and a KO/FR/EN adapter must not leak Russian. We compare two data curricula:

- **No-policy ablation.** Translation-style pair data only; no explicit examples binding active languages, JSON shape, age policy, or inactive-language constraints.
- **Policy+family repair.** Adds hand-authored exemplars covering the same *failure categories* the gates probe — active-language binding, JSON-only output, age-policy behavior, inactive-language refusal — but written on *different prompts* from the audit probes to avoid test-set leakage. This is data-side repair in the spirit of targeted augmentation (Sennrich et al., 2016), but the augmentation target is *deployment-state compliance*, not translation fluency.

The two curricula share the same translation-pair corpus and differ only in whether the policy/family slices are added. The no-policy mix contains 26,769 training rows (˜0.76M tokens); the policy+family mix adds policy and family exemplars on top of the same translation pairs and totals 58,476 rows (˜1.78M tokens), so policy+family sees more data overall. All runs share the base model, LoRA rank/alpha, learning rate, sequence length, step budget, decoding settings, common held-out loss set (1,200 examples), and the frozen 112-probe G1–G4 audit.

4.2 Main result

Table 4 gives the main result. The takeaway: loss alone tells a misleadingly favorable story. Both arms are in the same low-loss neighbourhood (no-policy 0.80, policy+family 0.67) and have similar G2; a curator who only ever sees the no-policy run — e.g. when the policy/family slice is silently dropped during data prep — has no signal that anything is wrong, because the loss looks like a normal fine-tuning result. Yet G3 and G4 are at 0%. No-policy collapses G3 schema and G4 session routing to 0%, while policy+family restores them to 95.8% and 91.7% on average. Held-out loss moves in the same direction (0.8040 $\rightarrow$ 0.6673; perplexities 2.23 and 1.95), and G2 is comparable across arms (within seed variance), so the difference is not general script capability — it is whether the curriculum contains the deployment states the interface will require. The two no-policy adapters illustrate the *false-green* pattern: their loss sits in the same low band as standard fine-tuning runs, yet they are RED on G3/G4. The point is not that loss *ranks* them above policy+family — policy+family is lower — but that loss alone gives no diagnostic signal: a curator who never trained policy+family would see only the no-policy loss, conclude the adapter is ready, and ship a deployment-broken artifact. We report the loss spread as a small-sweep consistency check, not a population variance estimate. An FP16-inference parity check on the seed-10 winner matches the bf16 numbers within rounding (G3=100, G4=100, G2=92.5), so the reported gate scores are not a precision artefact.

4.3 Curriculum decomposition

To check that both halves of the curriculum carry signal, we trained matched single-arm and combined-arm variants under the same 1500-step budget (Table 5). Policy-only fixes G3 (100%) but never opens G4 (0%); family-only opens G4 (83.3%) but is weaker on G3 (95.8%) and es-

Gate	GREEN	AMBER	RED
G2-52	total $\geq$50/52 *and* every direction $\geq$12/13	total $\geq$48/52 *and* every direction $\geq$10/13	below AMBER
G3-80	total $\geq$72/80 *and* every group $\geq$18/20	total $\geq$64/80 *and* every group $\geq$15/20	below AMBER

Table 3: Engineering triage thresholds. The deployment owner may set stricter or looser bands; the paper requires that the chosen version be logged alongside raw outputs and resulting action state.

Group	n	Loss	G1	G1 struct.	G1 age	G2	G3	G4
No-policy	2	0.8040±0.0038	12.5±17.7	35.4±50.1	68.8±26.5	92.5±0.0	0.0±0.0	0.0±0.0
Policy+family repair	3	0.6673±0.0001	55.6±6.4	62.5±18.2	93.1±12.0	91.7±1.4	95.8±7.2	91.7±14.4

Table 4: Four-language main-boost sweep. Means ± sample SD across seeds; gate values are percentages. G1 is the conjunction of structural and age-policy subchecks. **Read this way:** both arms reach low loss and similar G2, but G3 and G4 collapse to 0% under no-policy and recover to ∼92–96% under repair — the deployment-state gap that scalar selection cannot see.

Curriculum	G1	G2	G3	G4
no-policy (decomp. control, 1 seed)	41.7	85.0	20.8	0.0
policy-only	4.2	95.0	100	0.0
family-only	8.3	92.5	95.8	83.3
balanced repair	25.0	95.0	100	33.3
policy+family (main)	**55.6**	**91.7**	**95.8**	**91.7**

Table 5: Single-arm decomposition (gate pass-rate %; 3-seed mean for the main row, single-seed for ablation arms). The single-seed no-policy row is a decomposition control on a different seed/audit slice from the no-policy mean in Table 4; both confirm G3/G4 stay RED without policy/family. **Read this way:** policy-only recovers schema but not routing (G3=100, G4=0); family-only opens routing but is weaker on G3 and G1; only the combined recipe clears both G3 and G4 simultaneously.

pecially on G1 structure. The combined *policy+family* mix is the only curriculum that reaches high pass rates on both G3 and G4 simultaneously (G3 above 90%, G4 above 90%), while keeping G2 above 90%. Balanced and no-policy variants are reported as additional controls.

4.4 Cross-base-model replication

To check whether the recipe is Gemma-specific, we applied the policy+family curriculum to three other instruction-tuned base models under a comparable training budget (a single run per base, audited at an early checkpoint and at the final checkpoint of that run; the actual final checkpoint differs by base: checkpoint-774 for Qwen, checkpoint-1125 for Llama, checkpoint-765 for Phi, and two separate Gemma E2B control runs on different schedules (Ctrl A at ckpt-7310 and Ctrl B at ckpt-2000)). Gemma 4 E4B is reported as a stock-only control. Five stock baselines spanning four model families (Gemma 4 E2B and E4B, Qwen 2.5 3B, Llama 3.2 3B, Phi-3.5 mini) all fail G1/G3/G4 out of the box (0% on each), so the failure pattern is not a Gemma artefact — it is consistently observed across the stock instruction-tuned models we tested under this deployment interface. Under our curriculum, every newly repaired non-Gemma base (Qwen, Llama, Phi) recovers G3 to $\geq$91.7% at the final checkpoint; the Gemma E2B ctrl run on a separate save schedule is the exception (G3=45.8 at its final point) and is discussed below. Only Gemma 4 E2B (95.8/91.7 across 3 seeds) and Llama 3.2 3B (100/100, single run) also recover G4.

Per-base training dynamics. The early vs. final-checkpoint columns in Table 6 make four different trajectories visible. **Llama 3.2** already has G3=100% at the early checkpoint while G4 is only 4.2; by the final checkpoint G4 has risen to 100, so the bulk of the G4 gain happens between the two audited points. **Qwen 2.5** takes the opposite path: G3 climbs slowly (29.2 $\rightarrow$ 91.7) while G4 stays at zero throughout in this single run. **Phi-3.5** reaches G3=100 immediately and never opens G4 within the audited budget; we report this as an observation, not as evidence of an architectural limit. The two Gemma 4 E2B *ctrl* runs (separate save schedules) show a *G3 $\leftrightarrow$ G4 trade-off*: Ctrl A (ckpt-7310) reaches G3=95.8 but G4=0, while Ctrl B (ckpt-2000) reaches G4=41.7 but G3=45.8. The implication is that G3 (parseable JSON) and G4 (session-language routing) are not always learned together; while the same policy+family curriculum carries explicit exemplars for both states, the ctrl run on a separate save schedule lands on a G4-favoring solution that sacrifices G3, whereas the three main seeds settle on a region where both gates

Base	Variant	G1	G2	G3	G4
Gemma 4 E2B	stock	0.0	95.0	0.0	0.0
Gemma 4 E2B	PF (final)	55.6	91.7	95.8	91.7
Gemma 4 E4B	stock	0.0	95.0	0.0	0.0
Qwen 2.5 3B	stock	0.0	45.0	0.0	0.0
Llama 3.2 3B	stock	0.0	27.5	0.0	0.0
Phi-3.5 mini	stock	0.0	52.5	0.0	0.0
Qwen 2.5 3B	PF (early)	91.7	62.5	29.2	0.0
Qwen 2.5 3B	PF (final)	37.5	65.0	91.7	0.0
Llama 3.2 3B	PF (early)	37.5	57.5	100	4.2
Llama 3.2 3B	PF (final)	91.7	75.0	100	100
Phi-3.5 mini	PF (early)	0.0	67.5	100	0.0
Phi-3.5 mini	PF (final)	8.3	60.0	100	0.0
Gemma 4 E2B	ctrl A (ckpt-7310)	0.0	87.5	95.8	0.0
Gemma 4 E2B	ctrl B (ckpt-2000)	95.8	92.5	45.8	41.7

Table 6: Cross-base replication of policy+family repair (PF). For Qwen/Llama/Phi: "Early" is the first audited checkpoint of each run (170–250 steps); "final" is the last audited checkpoint (Qwen ckpt-774, Llama ckpt-1125, Phi ckpt-765). *Ctrl A* and *Ctrl B* are two separate Gemma 4 E2B control runs on different save schedules (longer and shorter), audited at the checkpoints shown. **Read this way:** every stock base scores 0% on G1/G3/G4 — failure is not Gemma-specific. The repair recipe lifts G3 to $\geq$91.7% on every newly repaired base (Qwen, Llama, Phi) by the final checkpoint; G4 is harder, recovered only on Llama and the main Gemma E2B seeds. The two Gemma E2B ctrl runs show the same curriculum can land on either a G3-strong or a G4-strong solution depending on schedule, illustrating that G3 and G4 are not learned by a single mechanism.

are at or near the upper band on the seed mean (with per-seed variation visible in Table 7).

4.5 Per-seed action trace

No free-form adapter is fully GREEN— this is the intended deployment distinction, not a claim of standalone model readiness. The raw adapter must satisfy content, JSON, and session routing without help; the deployment layer can additionally enforce JSON shape and active-language routing with deterministic templates or constrained decoding. Under this deployment-constrained interface, the policy+family curriculum yields two GREEN seeds and one AMBER seed. The no-policy ablation is less consistent: one seed becomes GREEN, the other remains RED because its age-policy behavior fails. The practical recipe is not "LoRA alone solves the four-language case." The shipped artifact is a *four-language LoRA plus a thin deployment layer* enforcing JSON shape and active-language routing with deterministic templates or constrained decoding: the LoRA carries content/script behaviour that templates cannot synthesise; the deployment layer carries the structural states (G3, G4) that LoRA-only training does not reliably learn. The gates make this split principled rather than ad hoc by naming each state and recording where the responsibility for satisfying it sits.

Variant	Loss	G3/G4	Free	Deployed
PF-09	0.6672	100/75	AMBER	AMBER
PF-10	0.6673	100/100	RED	GREEN
PF-11	0.6673	87.5/100	RED	GREEN
NoPol-10	0.8067	0/0	RED	GREEN
NoPol-11	0.8014	0/0	RED	RED

Table 7: Per-seed action trace for the four-language sweep. "Free" is raw free-form generation. "Deployed" applies deterministic JSON/session constraints and leaves content/script behavior to the adapter. The deployment layer can rescue interface debt, but policy+family is lower-loss and more consistent across the seeds tested.

5 Discussion

The strongest result is not the discovery trajectory; it is the repair loop. Once schema, script, and session-routing states are written as gates, the same states define a curriculum, an evaluation target, and a promotion decision. Compared to post-hoc rejection, this connects an audit failure to a concrete retraining action rather than discarding the run.

Cross-base evidence is consistent with the failure being an *interface* property rather than a base-model bug: every stock instruct model tested scores 0% on G1/G3/G4, and three repaired bases (Qwen, Llama, Phi) all recover G3 to $\geq$91.7%. We do not overclaim: every base failing an app-specific schema is partly an artefact of the schema, and only Llama and the main Gemma E2B seeds also recover G4. What the cross-base block supports is that the recipe is not Gemma-specific and that G3 and G4 are not learned by one mechanism (see the two Gemma E2B ctrl runs). The no-policy ablation makes the failure class concrete: under matched training settings (same base, hyper-parameters, audit, translation corpus; differs only in the added policy/family slice (so PF sees more data)), removing the policy and family exemplars produces an adapter with low held-out loss whose G3/G4 collapse to 0% — a false-green that scalar selection alone cannot detect.

Reproducibility. Each central number is tied to a machine-readable artifact: per-gate score

JSONs with raw generations, audit and loss JSONs for the repair sweep, deployment-constraint summaries, and the scorer scripts that produced them. We retain raw outputs because they reveal whether a failure is wrong script, mixed script, non-JSON prose, missing required keys, wrong field types, or inactive-language leakage — exactly the distinctions a downstream parser cares about. The training pipeline and audit scripts are released at `https://github.com/carryer123/gemma4-trilingual-family` under Apache 2.0 (code) and CC-BY 4.0 (audit protocol and probe set). The intended user is a small deployment team, not a leaderboard organiser; the workflow is summarised in Figure 1.

6 Limitations

The main four-language sweep is small: three policy+family seeds and two no-policy seeds on Gemma 4 E2B under one deployment specification and one frozen audit suite. The cross-base replication is a single run per base (Qwen 2.5 3B, Llama 3.2 3B, Phi-3.5 mini) at a comparable training budget, so it bounds the failure pattern but does not estimate variance across seeds for those bases. G2 checks script-state compliance rather than phonetic quality, G3 checks schema shape rather than semantic correctness, and deployment-constrained GREEN states depend on deterministic interface guards. Gate thresholds are heuristics: we report sensitivity but do not calibrate precision or recall against a ground-truth deployment success label. The natural next step is a larger repair study across more seeds, model families, language pairs, and stricter schema modes. The gates also do not test adversarial robustness; prompt-injection and jailbreak attacks on a deployed multilingual interface (Liu et al., 2023) are out of scope and require their own threat-model-specific evaluation.

7 Conclusion

We argue that deployment gates should do more than reject adapters after the fact. When a multilingual deployment depends on script state, schema validity, age-policy behaviour, and session routing, those states can define both promotion criteria and repair data. In the four-language KO/RU/FR/EN sweep, a gate-aware policy+family curriculum brings G3 from 0% to 95.8% and G4 from 0% to 91.7% over a no-policy ablation (matched on base, hyper-parameters, audit, and translation corpus but not on data volume), with no held-out loss penalty. The same curriculum recovers G3 to $\geq$91.7% on three additional base models (Qwen 2.5, Llama 3.2, Phi-3.5), so the failure pattern is a property of the deployment interface, not of any one base. The contribution is a bounded, reproducible promotion-and-repair loop for multilingual LoRA adapters.

References

Emily M. Bender and Batya Friedman. 2018. Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics*, 6:587–604.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, and 1 others. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.

Tim Bray. 2017. The javascript object notation (json) data interchange format. Technical Report 8259, RFC Editor.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of ACL*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.

Gemma Team. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc'Aurelio Ranzato, Francisco Guzmán, and Angela Fan. 2022. The flores-101 evaluation benchmark for low-resource and multilingual machine translation. *Transactions of the ACL*, 10:522–538.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt.

2021. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Daniel Khashabi, Sewon Min, Tushar Khot, Ashish Sabharwal, Oyvind Tafjord, Peter Clark, and Hannaneh Hajishirzi. 2020. UnifiedQA: Crossing format boundaries with a single qa system. In *Findings of EMNLP*.

Tom Kocmi, Eleftherios Avramidis, Rachel Bawden, Ondřej Bojar, Anton Dvorkovich, Christian Federmann, Mark Fishel, Markus Freitag, Thamme Gowda, Roman Grundkiewicz, and 1 others. 2024. Findings of the WMT24 general machine translation shared task: The LLM era is here but MT is not solved yet. In *Proceedings of the Ninth Conference on Machine Translation*, pages 1–46.

Julia Kreutzer, Isaac Caswell, Lisa Wang, Ahsan Wahab, Daan van Esch, Nasanbayar Ulzii-Orshikh, Allahsera Tapo, Nishant Subramani, Artem Sokolov, Claytone Sikasote, and 1 others. 2022. Quality at a glance: An audit of web-crawled multilingual datasets. *Transactions of the ACL*, 10:50–72.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, and 1 others. 2023. Holistic evaluation of language models. *Transactions on Machine Learning Research*.

Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, and 1 others. 2022. Few-shot learning with multilingual generative language models. In *Proceedings of EMNLP*.

Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. 2023. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. Model cards for model reporting. In *Proceedings of FAT**.

NLLB Team. 2022. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of ACL*.

Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. 2020a. Adapterhub: A framework for adapting transformers. In *Proceedings of EMNLP: System Demonstrations*.

Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, and Sebastian Ruder. 2020b. Mad-x: An adapter-based framework for multi-task cross-lingual transfer. In *Proceedings of EMNLP*.

Maja Popović. 2015. chrf: character n-gram f-score for automatic mt evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*.

Matt Post. 2018. A call for clarity in reporting bleu scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of nlp models with checklist. In *Proceedings of ACL*.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of EMNLP*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Improving neural machine translation models with monolingual data. In *Proceedings of ACL*.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, and 1 others. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*.

Ahmet Üstün, Viraat Aryabumi, Zheng-Xin Yong, Wei-Yin Ko, and 1 others. 2024. Aya model: An instruction finetuned open-access multilingual language model. *arXiv preprint arXiv:2402.07827*.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of EMNLP Workshop BlackboxNLP*.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.